Time to Split: Exploring Data Splitting Strategies for Offline Evaluation of Sequential Recommenders

Danil Gusak*
AIRI
Moscow, Russian Federation
Skoltech
Moscow, Russian Federation
danil.gusak@skoltech.ru

Anna Volodkevich*
Sber AI Lab
Moscow, Russian Federation
Skoltech
Moscow, Russian Federation
volodkanna@yandex.ru

Anton Klenitskiy*
Sber AI Lab
Moscow, Russian Federation
antklen@gmail.com

Alexey Vasilev
Sber AI Lab
Moscow, Russian Federation
HSE University
Moscow, Russian Federation
alexxl.vasilev@yandex.ru

Evgeny Frolov
AIRI
Moscow, Russian Federation
HSE University
Moscow, Russian Federation
frolov@airi.net

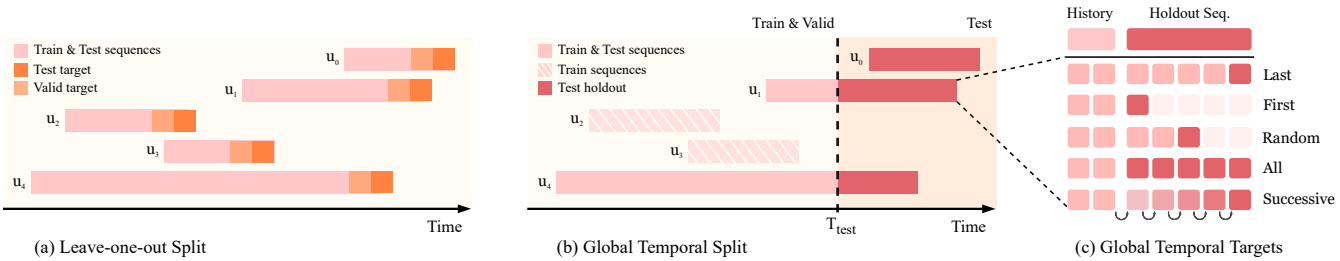


Figure 1: Data splitting and target selection strategies for sequential recommendations. (a) Leave-one-out split. (b) Global temporal split: all interactions after timepoint T_{test} are placed in the holdout set, targets for these holdout sequences are chosen according to (c). (c) Target items selection options for each holdout sequence (applicable for both test and validation sequences).

Abstract

Modern sequential recommender systems, ranging from lightweight transformer-based variants to large language models, have become increasingly prominent in academia and industry due to their strong performance in the next-item prediction task. Yet common evaluation protocols for sequential recommendations remain insufficiently developed: they often fail to reflect the corresponding recommendation task accurately, or are not aligned with real-world scenarios.

Although the widely used *leave-one-out* split matches next-item prediction, it permits the overlap between training and test periods, which leads to temporal leakage and unrealistically long test horizon, limiting real-world relevance. *Global temporal splitting* addresses these issues by evaluating on distinct future periods. However, its applications to sequential recommendations remain loosely defined, particularly in terms of selecting target interactions and constructing a validation subset that provides necessary consistency between validation and test metrics.

In this paper, we demonstrate that evaluation outcomes can vary significantly across splitting strategies, influencing model rankings and practical deployment decisions. To improve reproducibility in both academic and industrial settings, we systematically compare different splitting strategies for sequential recommendations across multiple datasets and established baselines. Our findings show that prevalent splits, such as leave-one-out, may be insufficiently aligned with more realistic evaluation strategies.

CCS Concepts

• **Information systems** → **Recommender systems**.

Keywords

recommender systems; sequential recommendations; data splitting

ACM Reference Format:
Danil Gusak, Anna Volodkevich, Anton Klenitskiy, Alexey Vasilev, and Evgeny Frolov. 2025. Time to Split: Exploring Data Splitting Strategies for Offline Evaluation of Sequential Recommenders. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25), September 22–26, 2025, Prague, Czech Republic.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3705328.3748164

*Authors contributed equally to the paper

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RecSys '25, Prague, Czech Republic

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1364-4/25/09
https://doi.org/10.1145/3705328.3748164

1 Introduction

Sequential recommender systems (SRS) have become a prominent choice for the next-item prediction (NIP) task [68, 71]. By modeling each user's interaction history as an ordered sequence, sequential approaches can effectively capture temporal patterns and incrementally update user representations without retraining [29, 56].

One critical component in the experimental pipeline for recommender systems (RS) is data splitting. Previous research highlighted the sensitivity of recommendation outcomes to different splitting strategies in classical recommender scenarios [28, 41, 55]. However, despite rapid advances in sequential architectures, evaluation protocols for SRS remain insufficiently developed: researchers commonly rely on leave-one-out (LOO) split that *violate the global timeline of user-item interactions* [23, 28], or simply adopt global temporal split (GTS) from classical top-K recommendation task (e.g. 80/10/10 temporal split [34, 60]), *without tailoring it to next-item prediction setting* (see Section 3.0.2). These mismatched protocols raise concerns for both the task alignment and the real-world relevance of offline evaluation results.

In this paper, we seek to close this gap and systematically examine various global temporal splitting variants *specifically tailored to sequential recommendation scenarios*. We formalize and compare the prevalent LOO approach with GTS-based splits, defining the options for ground-truth target selection suited to the next-item-prediction task. Some of these options, appeared in recent research [14, 19, 31], include such NIP-oriented targets as *the user's last interaction in a time-separated holdout sequence* or *successive target*, where each subsequent interaction of the holdout sequence is treated as a separate target with incrementally extended input history.

Furthermore, we investigate different validation schemes for GTS, including *global temporal, user-based, and last training item* splitting approaches, to analyze the trade-off between training data amount, data recency, and presence of temporal leakage in validation. We examine the resulting subsets obtained after different splits to assess training set sizes, number of test users, durations of test and validation periods, and time-gap biases for GTS targets.

Our extensive experiments on multiple datasets and widely-used sequential models show that the choice of data splitting strategy for SRS can *significantly impact evaluation metrics and model rankings*.

In summary, *our main contributions* are:

- We explore different global temporal split variants for SRS, distinguished by choice of ground-truth targets and validation set construction, and compare them to leave-one-out split, highlighting their properties, advantages, and disadvantages;
- We systematically analyze metric correlations and consistency in final model rankings for different splits, identifying which strategies better align with real-world scenarios and the next-item prediction task;
- We evaluate different validation schemes for GTS and identify those that offer reliable model selection.

2 Related Work

In this section, we first outline the evolution of SRS and then review existing studies on data splitting strategies.

Sequential Recommenders. Early SRS research was advanced with gated RNNs [24, 26], which outperformed MC models [20, 21]. The emergence of Transformers [62] led to further improvements, consistently surpassing prior methods [29, 56]. Subsequent development is continuing in various directions, including enriching recommendation models with side information [35, 37, 48], integrating contrastive learning approaches [6, 9, 47, 70], and modifying the self-attention mechanism [4, 10]. Recently, generative recommendations [36, 49, 54, 64] and integration of SRS with large language models [5, 13, 67, 69] have emerged as promising directions.

While some works redesign training objectives in favor of a long-term engagement [44], the next-item prediction task remains dominating in the vast majority of the works mentioned above. Given the rapid advancement and widespread use of sequential recommenders, it is critical to carefully select data splitting strategies, matching the real-world usage and recommendation task, for robust evaluation and comparison of the SRS models.

Data Splitting Strategies. Reproducible evaluation remains challenging in SRS research: only a small fraction of new algorithms reliably outperform rigorously optimized baselines [11]. Recent works also show that variations to experimental protocols can reshuffle model rankings [23].

Data splitting, an essential part of the evaluation protocol, has been deeply analyzed in various studies. Some works explore the impact of different evaluation settings, including data splitting, on the performance of top-N recommendation algorithms [58, 72]. Other studies provide a critical analysis of data leakage in commonly used random and LOO splits [28, 55]. Further, recent works argue that a better evaluation strategy should use a global timeline to avoid data leakage and better reflect real-world scenarios [28, 55]. Meng et al. [41] quantify how splitting choices can reorder leaderboards with Kendall's correlation and note that certain splits benefit specific models. Several other studies consider different aspects of data splitting: evaluating models on a sequence of consecutive time-based splits to account for how model performance changes over time [52]; highlighting how the length of the training window influences performance [63]; and investigating how randomness in data splitting affects the variability of metrics [65].

Unlike the works mentioned above, we focus on sequential recommendation algorithms and splitting strategies tailored for the next-item prediction task, where we observe an absence of a unified split protocol that both prevents data leakage and ensures reproducible, fair comparisons of SRS in a close-to-real usage setting.

3 Data Splitting Strategies for SRS

3.0.1 Important splitting properties considered. To be aligned with the real-world usage and avoid data leakage from the future, the split must preserve a *global timeline* [28], meaning that any interaction occurring after the timepoint T_{test} is excluded from training. In production, sequential recommenders are often used in online scenarios for *next-item prediction* over some period (e.g., day or week) before model retraining, and *all previous user history is available for inference*, including interactions after training cutoff. A splitting strategy not aligned with real-world usage can inflate offline performance and promote models that underperform in production.

Statistical tests are often used to make an offline evaluation result a reliable estimate of a model's performance [27]. Common methods like the paired Student's t-test become more sensitive as

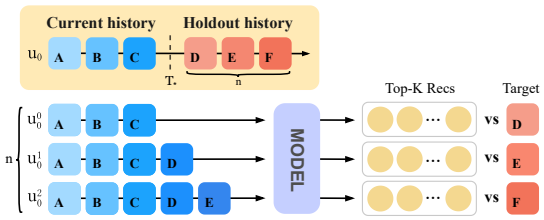


Figure 2: Successive evaluation scheme applied to one user with $n = 3$ holdout interactions.

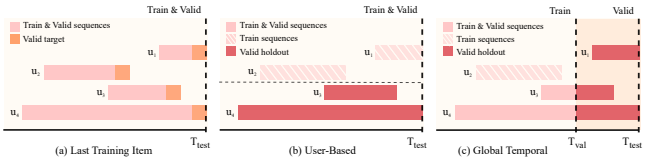


Figure 3: Validation split options for GTS (Fig. 1b): (a) each user's Last training item is a target, (b) User-based: interactions of n random users are reserved for holdout, (c) Global temporal: interactions after T_{val} are reserved for holdout. Targets for holdout sequences are chosen according to Figure 1c.

the number of test users increases [8]. Thus *number of test users after data splitting should be sufficient* to provide a robust estimate and draw reliable conclusions.

3.0.2 Splitting strategies for SRS in publications and RS frameworks. To assess the prevalence of data splitting practices, we analyzed recent SRS papers from conferences RecSys, SIGIR, and CIKM for 2022–2024. Out of 75 papers that conducted offline evaluation, 77.3% used LOO-based splits, 16% used GTS-based strategies, and only 6.7% of works applied GTS in the next-item-prediction setting. This shows that splits, both preserving the global timeline and NIP task-oriented [14, 19, 31, 42], remain rare within the RS community.

We also observe an absence of splits suitable for the SRS in popular RS frameworks. Table 1 shows that frameworks often offer LOO and GTS-based splits, but lack direct support for their combinations tailored for NIP with GTS. The absence of appropriate splits in widely adopted frameworks limits their usage in RS community.

3.0.3 Splitting strategies for SRS in detail. In our work, we consider the leave-one-out split as the most popular in sequential recommendation research, and the global temporal split with the different target variants, either widespread or aligned with the next-item prediction task. We leave out of scope the other splitting strategies, based on the random, temporal user, or user split defined by Meng et al. [41], as they are less common for sequential recommendations, mismatch the task, and fail to preserve the global timeline.

- **Leave-one-out (LOO)** holds out each user's final interaction for testing and the second-to-last for validation (Fig. 1a). LOO supports local chronology, aligns with the NIP task, and in some cases maximizes the amount of training data and the number of test users. LOO is used in fundamental works [29, 56]. However, LOO ignores a global timeline, allowing future data to leak into the training set [23, 55], and produces an unrealistically long test period, as target events span the entire dataset.
- **Global Temporal Split (GTS)** defines a global timepoint (cutoff) T_{test} based on a quantile of interaction data or on a required number of test sequences, and assigns all interactions after T_{test} to the holdout set (Fig. 1b). The GTS description for sequential recommendation is incomplete without specifying the target item selection strategy. Nevertheless, the general properties of GTS are as follows: GTS completely prevents data leakage and gives control over the test period duration. Compared to LOO, GTS typically produces fewer test users, and it often requires a dataset-specific choice of a global timepoint to balance the number of test users, test period duration, and amount of training data.

Table 1: Splits available in popular RS frameworks

Split Type	Cornac [51]	DaisyRec [57]	Elliot [1]	FuxiCTR [74]	Recomdrs. [15]	RecBole [73]	RecPack [43]	RePlay [61]
LOO-based	✓	✓	✓	✗	✓	✓	✓	✓
GTS-based	✗	✓	✓	✓	✓	✓	✓	✓
GTS + NIP	✗	✗	✗	✗	✗	✗	✓	✗

Global temporal split target options. We propose to consider a wide range of target options for GTS illustrated in Figure 1c. The common approach bundles *all of a user's holdout items* as a single ground-truth set [16], which we refer to as an **All** target. The other considered targets are based on the following: *the selected holdout item (or each of the holdout items) is treated as a separate target. All interactions that occurred before the current target are available during inference*, regardless of whether they occur before or after the global split point, and *all subsequent interactions are discarded.* We consider the following target-selection strategies aligned with the next-item prediction task:

- **Last**: the last holdout interaction is considered as a target. This target is an intuitive combination of LOO and GTS [17, 19, 31]. This target emphasizes later post-timepoint behavior.
- **First**: the first holdout interaction is considered as a target. Emphasizes early post-timepoint behavior and may be skewed due to session-boundary artifacts (see Section 4.2).
- **Successive (Sucv.)**: each holdout interaction is a separate target (Fig. 2) [14, 55]. As a result, for n holdout interactions, we obtain n separate metric evaluations. Then the metrics could be preliminarily averaged by user or directly used to estimate the mean metric value. Preliminary averaging by the user is more aligned with real-world usage, with each user being equally important, avoiding distribution bias towards active users who produce a significant number of interactions. Successive target is the most aligned with the real-world SRS usage in the online scenario and represents an average performance after a global timepoint. Yet it is more complex to implement, and the average holdout sequence length linearly increases the inference computation compared to variants with one target item per sequence.
- **Random**: target interaction is randomly sampled from the holdout sequence. It provides a proxy for an average post-cutoff performance and can be considered an alternative to successive evaluation. However, due to its non-deterministic nature, ensuring reproducibility requires extra actions like averaging over multiple random seeds or publishing sampled targets.

Validation splitting strategies for GTS. The most straightforward validation approach for GTS is global temporal validation. However, we also consider the other validation options that allow for the inclusion of more recent data before the test global timepoint

into the training dataset. It provides better quality on the test subset without model retraining on the training and validation data combination. Although those options permit some temporal leakage into validation set, the final model ranking remains unaffected, since the ultimate evaluation metric is computed strictly on the GTS-based test set. We consider three validation options for GTS (Fig. 3):

- **Global Temporal (GT)** sets a cutoff T_{val} before T_{test} and holds all interactions after T_{val} for validation. It prevents temporal leakage; matches test split, but shrinks the training set and drops recent user interactions.
- **Last Training Item (LTI)** holds each user's final interaction before T_{test} as the validation target. It covers all users and aligns with NIP but yields an unrealistically long validation period.
- **User-Based (UB)** reserves the entire histories of a random subset of users for validation holdout. It preserves full histories for training users and limits training data reduction. Still, it yields an unrealistically long validation period and requires direct control of the number of users in training and validation.

For the UB and GT validation splitting strategies, there is a need for the target item selection, and options considered in *Global Temporal Split Target Options*, especially the Last, Successive, and Random for UB, are also applicable for the validation subset.

Reporting of splitting strategy details. The necessity of careful description of splitting details is highlighted in multiple works on the topic [16, 41]. In addition to details common to various splits and recommendation tasks (such as including the train-test split ratio and cold items filtering), we emphasize the need to report specific details for the sequential recommendation task. Those details depend on the chosen splitting strategy and may include the target item or item set selection, the input sequence building approach for selected targets, the presence of new sequences started after the global timepoint in holdout, and the target selection approach for these sequences. In Section 4.0.2, we report details of the GTS splitting used in this study.

4 Experiments

We design experiments to answer the following research questions:

RQ1 What are the important properties of subsets obtained with different splitting strategies?
RQ2 What is a distribution of time delta between consecutive user interactions, and how does it affect target item selection for GTS?
RQ3 How consistent are recommendation metrics for different splitting strategies in terms of correlation?
RQ4 How do different data splitting strategies influence the final model rankings?
RQ5 Which validation strategies are more appropriate for GTS?
RQ6 How does retraining the model on the combined training and validation data influence its final test performance?

The code for our experiments is available in the repository.[1]

4.0 Experimental Setup

4.0.1 Datasets. We conduct our experiments on eight popular real-world datasets, mostly selected for their strong sequential structure, as highlighted in recent research [31]: Amazon Reviews [40] datasets (**Beauty**, **Sports**), MovieLens-1M (**ML-1M**) and MovieLens-20M (**ML-20M**) [18], BeerAdvocate (**BeerAdv**) [39], **Diginetica**[2], **YooChoose** [2], and **Zvuk** [53]. To manage computational costs while ensuring sufficient data for analysis, we sample 2,000,000 users from the YooChoose dataset and 20,000 users from Zvuk.

Table 2: Statistics of the datasets after preprocessing

Dataset	#Interact.	#Users	#Items	Avg. Len.	Density (%)	#Days
Beauty [40]	198 502	22 363	12 101	8.9	0.07	4 424
BeerAdv [39]	1 475 412	14 635	22 074	100.8	0.46	5 620
Diginetica[2]	485 903	61 279	25 593	7.9	0.03	152
ML-1M [18]	999 611	6 040	3 416	165.5	4.84	1 038
ML-20M [18]	19 984 024	138 493	18 345	144.3	0.79	7 385
Sports [40]	296 337	35 598	18 357	8.3	0.05	4 521
YooChoose [2]	2 792 229	335 203	20 758	8.3	0.04	181
Zvuk [53]	8 087 953	19 267	150 206	419.8	0.28	91

Consistent with prior studies [30, 59], we treat any review or rating as implicit feedback. Additionally, following common practices [12, 50, 58], we apply p-core filtering with p equal 5 to discard unpopular items and short user sequences. Furthermore, we eliminate consecutive repeated items in user interaction histories [23]. Table 2 summarizes the final statistics of the datasets.

4.0.2 Evaluation. For GTS, we use $q_{0.9}$ interaction quantile to conduct the main experiments. We filter out sequences of length one from all data subsets. For test and validation subsets, we use all sequence elements before the target item as an input in inference, regardless of their position relative to the global timepoint. For sequences started after the global timepoint, we excluded the first item from the targets to provide a model with at least one element of the sequence. For the same reason, we only use sequences that start before the global timepoint for the All target. We apply preliminary metric averaging within a sequence (user history) for the Successive target. For GTS with GT and UB validation, we use the Last target as a reasonable and deterministic choice, allowing to reduce the training computational costs compared to the Successive target. For GTS with UB validation, we sample 1024 users.

Recent studies highlighted the limitations of using sampled metrics for evaluating RS, as they can introduce biases and misrepresent model performance [3, 7, 32]. Following best practices, we use popular *unsampled* top-K ranking metrics[3]: Normalized Discounted Cumulative Gain (NDCG@K), Mean Reciprocal Rank (MRR@K) and HitRate (HR@K), with K = 5, 10, 20, 50, 100.

In line with common practices, we also apply a *filter seen* [25] step, removing items from the recommendation lists that users have already interacted with. This step is applied to all datasets except Zvuk, YooChoose, and Diginetica, as these datasets naturally contain repeated user-item interactions [31].

4.0.3 Models. We conduct our experiments using three popular sequential recommender system baselines: **SASRec**$^+$ [30], an adaptation of the original PyTorch implementation[4] that employs full cross-entropy loss (CE) over the entire item catalog to achieve state-of-the-art performance [30, 46]; **BERT4Rec** [56], an efficient implementation[5] using the Transformers library [66]; and **GRU4Rec** [24] implementation[5] with full CE loss [30].

[1] https://github.com/monkey0head/time-to-split

[2] https://competitions.codalab.org/competitions/11161

[3] We compute metrics using RePlay framework: https://github.com/sb-ai-lab/RePlay

[4] https://github.com/pmixer/SASRec.pytorch

[5] https://github.com/antklen/sasrec-bert4rec-recsys23

Table 3: Holdout statistics for different splits ($q_{0.9}$ for GTS)

Set	Split	Stats.↓	Beauty	BeerAdv	Diginetica	ML-1M	ML-20M	Sports	YooChoose	Zvuk
Full Data	–	#Days	4,424	5,620	152	1,038	7,385	4,521	181	91
		Lifetime (%)	12.4	11.56	0.01	9.14	2.66	12.0	0.01	43.47
		#Users	22,363	14,635	61,279	6,040	138,493	35,598	335,203	19,267
		Seq. Len.	8.88	101	7.93	166	144	8.32	8.33	420
Valid	LOO	#Days (%)	84.0	66.9	100	100	94.7	69.4	100	100
		#Users (%)	100	100	100	100	100	100	100	100
	GT	#Days (%)	1.38	2.76	6.58	2.41	10.9	1.50	8.29	7.69
		#Users (%)	28.0	35.4	9.55	17.3	11.9	27.2	8.87	41.7
		Holdout Len.	2.84	25.6	7.47	86.0	109	2.73	8.45	90.7
	UB	#Days (%)	45.6	57.3	90.1	23.8	79.3	41.2	90.6	91.2
		#Users (%)	4.58	7.00	1.67	17.0	0.74	2.88	0.31	5.31
	LTI	#Days (%)	82.4	63.7	94.1	23.8	79.6	66.1	90.6	91.2
		#Users (%)	96.0	94.0	90.0	99.5	90.2	96.1	90.3	95.6
Test	LOO	#Days (%)	84.0	66.9	100	100	94.5	68.1	100	100
		#Users (%)	100	100	100	100	100	100	100	100
	GTS	#Days (%)	1.60	3.26	5.92	76.1	14.9	1.95	9.39	8.79
		#Users (%)	27.3	35.0	10.4	20.0	13.4	28.7	9.74	43.8
		Holdout Len.	3.25	28.8	7.66	82.7	108	2.89	8.55	95.9

4.0.4 Implementation Details. In our experiments, we define wide ranges for each model's hyperparameters [22, 30, 45]. For both SASRec$^+$ and BERT4Rec, we vary the hidden sizes between 32 and 256, use between 1 and 3 self-attention blocks, and from 1 to 4 attention heads. We applied a masking probability of 0.2 for BERT4Rec. In the case of GRU4Rec, we explore hidden sizes from 16 up to 512 and vary the number of GRU layers from 1 to 4. We also employ dropout rates between 0.1 and 0.5 across all models.

For all models, we use a training batch size of 256 and set the maximum sequence length to 128. We train the models using the Adam optimizer with a learning rate of 10^{-3} and set the maximum number of epochs to 300. During training, we monitor NDCG@10 on the validation set to control model convergence through the early stopping mechanism. Specifically, we set the patience parameter to 10 epochs for SASRec$^+$ and GRU4Rec, while for BERT4Rec we use a patience of 20 to accommodate its slower convergence, observed in prior studies [30, 45]. All experiments are conducted on NVIDIA H100 GPUs with 80GB HBM3 memory.

4.1 Split statistics and properties (RQ1)

Different splitting strategies generate different training, test, and validation subsets, which could vary significantly. In this section, we analyze important subsets' properties and the influence of the splitting strategy on them.

4.1.1 Amount of training data left. GTS offers *direct control of the amount of training data*; thus, for the 0.9-quantile, 90% of data points are left for training and validation. It seems intuitive that the LOO split should leave more data for training, as it does not preserve the global timeline and holds only two items of each sequence. However, for the datasets with short sequences (Beauty, Sport, Diginetica, YooChoose), LOO leaves about 75% of interactions for training, less than any GTS variant. It should be noted that *splitting also affects average sequence length.* Thus, for the datasets with a long user lifetime, calculated as a median period of user activity divided by the dataset time period (Table 3, Lifetime (%)), GTS shortens training sequences by roughly 20% in Beauty, Sports and Zvuk, while lengths in Diginetica and YooChoose remain nearly unchanged.

4.1.2 Trade-off between the number of test users, the volume of training data, and the duration of the test period. Table 3 shows that LOO includes 100% of users in the test set, whereas GTS at $q_{0.9}$ covers only 10%–44%. Thus, it could be easier to obtain statistically significant results with LOO, but those results are obtained in an unrealistic setup, not preserving the global timeline. Table 4 compares GTS across quantiles $\{0.8, 0.9, 0.95, 0.975\}$, revealing up to a 4 times decrease in test users at $q_{0.975}$. Lower quantiles raise user counts but extend the test subset duration, making it unrealistically long. Holdout period for GTS at $q_{0.9}$ spans 2%–15% of the timeline (8–1,100 days) and rises to 100% under LOO (7,385 days for ML-20M). For Zvuk and Diginetica, GTS at $q_{0.9}$ yields nearly a week-long test with thousands of users, matching real usage time period and users sufficiency requirements.

Table 4: Test subset statistics for GTS for different quantiles

Dataset	Len.	Holdout Len.				#Users (K)					#Days				
	Full	$q_{0.8}$	$q_{0.9}$	$q_{0.95}$	$q_{0.975}$	*Full*	$q_{0.8}$	$q_{0.9}$	$q_{0.95}$	$q_{0.975}$	*Full*	$q_{0.8}$	$q_{0.9}$	$q_{0.95}$	$q_{0.975}$
Beauty	8.88	3.88	3.25	2.76	2.45	22.4	10.2	6.11	3.52	1.91	4,424	138	71	35	19
BeerAdv	101	42.5	28.8	18.7	12.0	14.6	6.94	5.12	3.94	3.07	5,620	354	183	94	48
Diginetica	7.93	7.68	7.66	7.38	6.55	61.3	12.7	6.35	3.29	1.86	152	20	9	4	2
ML-1M	166	112	82.7	61.5	45.6	6.04	1.78	1.21	0.81	0.55	1,038	818	790	617	400
ML-20M	144	126	108	92.8	86.9	139	31.7	18.6	10.8	5.75	7,385	1,994	1,100	569	201
Sports	8.32	3.52	2.89	2.61	2.60	35.6	16.7	10.2	5.63	2.79	4,521	163	88	43	22
YooChoose	8.33	8.49	8.55	8.57	8.79	335	65.8	32.7	16.3	7.94	181	34	17	10	5
Zvuk	420	150	95.9	61.6	42.8	19.3	10.8	8.43	6.57	4.73	91	16	8	4	2

The holdout length per user is also affected by the split. For LOO, it is always equal to one, while GTS holdout length varies with user lifetime and quantile. Table 4 reports holdouts exceeding 100 items in Zvuk and ML-20M, which significantly increases inference cost for successive evaluation. However, even for the higher quantiles or specific datasets like ML-1M, we observe an unrealistically long test period in some cases, combined with a lack of users. We recommend using datasets and GTS quantiles that balance user count, test duration, and training data amount.

4.1.3 Influence of validation type on validation and training subsets properties. Table 3 shows that test and validation subsets for LOO and for GTS with global temporal validation yield aligned holdout durations and user shares, thus those strategies could provide better validation and test metrics compliance. In contrast, the LTI and UB validations yield subsets of a long validation period, less aligned with GTS test set statistics. Since UB reserves a user subset, its size should be additionally controlled to balance the training data amount and the number of validation users.

4.2 Time gaps for different targets in GTS (RQ2)

4.2.1 Temporal distribution of user activity inside sequence. Experimenting with real-world and some academic datasets, we observed that the First interaction after the global timepoint as the GTS target yields lower metrics than for subsequent items. We hypothesized that the global timepoint often hits a period of user's inactivity, an inter-session period, and thus the First item becomes the beginning

Table 5: Median delta δ (in seconds) between each target interaction and the previous one: for different (a) validation types on the validation, and (b) target options on the test set

Set	Setup↓	Beauty	BeerAdv	Diginetica	ML-1M	ML-20M	Sports	YooChoose	Zvuk
Full Data	–	345,600	73,182	58	0	11	172,800	59	14
(a) Valid	LOO	172,800	360,900	63	18	17	86,400	59	98
	GT Last	1,036,800	446,371	71	27	41	1,209,600	67	84
	UB	604,800	691,188	70	15	19	518,400	65	78
	LTI	604,800	690,794	70	15	21	518,400	65	68
(b) Test	LOO	604,800	737,140	70	17	20	518,400	65	73
	Last	1,382,400	508,452	70	67	29	1,296,000	68	91
	First	8,640,000	4,921,729	186	7,153,214	21,145,894	11,577,600	259	346,010
	Rand.	3,628,800	439,805	65	35	15	4,752,000	62	120
	Succ.	172,800	75,916	58	22	14	86,400	60	67

of the next user session. In our work, we do not explicitly identify sessions, as it is often a matter of professional judgment (heuristic) [27], but in Table 5, we report median time gaps between all consecutive interactions in datasets (indicated as Full Data) and each target and its previous event. The gap for the First target is much larger than for other targets, and the time gap across the dataset, which makes this target biased. Thus, we do not recommend using the First item after the global timepoint as a target.

4.2.2 Time gap patterns across targets. Table 5 shows that some review datasets (Beauty, Sports, BeerAdvocate) have day-level median between-interactions time gaps, while the other datasets have second- or minute-level gaps that remain consistent across validation setups and targets except the First. On Beauty and Sports, short holdouts could lead to selection of the same (first) item as First, Last, and Random target, inflating the gap for Last and Random targets. As shown in Figure 4, which plots the log-scaled gap densities for Zvuk, the First target distribution is shifted right, while other targets match the overall pattern. This confirms that, except for the First interaction, all targets after the global timepoint are appropriate in terms of temporal intervals between interactions.

4.3 Consistency between different splits (RQ3)

Absolute metric values can vary significantly across different splitting strategies, making direct comparison of the results impossible. However, if the relative ranking of models is preserved across splits, then conclusions about their comparative performance remain consistent. To analyze the agreement between pairs of splits, we compute the correlation between metrics obtained on these splits across different models and hyperparameter settings. We treat the GTS with Successive target as the most realistic and closest to production use, and compare all other splits against it, suggesting that an appropriate split for next-item prediction should exhibit high correlation with this reference.

We train the models with a wide range of hyperparameters defined in Section 4.0.4, resulting in 108 configurations for SASRec$^+$ and BERT4Rec, and 104 for GRU4Rec. Using multiple hyperparameter settings for each model allows us to generate a large number of evaluation points, leading to more statistically robust conclusions. For a comprehensive analysis, we consider multiple evaluation metrics (HR, MRR, NDCG) at different values of K. To assess the agreement between the metrics obtained from different splits, we use Kendall and Spearman rank correlation coefficients.

4.3.1 Visual scatterplot analysis. The first step is a visual analysis of scatter plots for different pairs of splits. Figure 5 shows pairwise comparisons between GTS Successive and other splits for

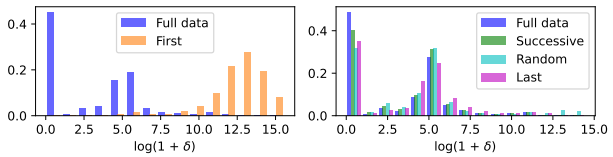


Figure 4: Distribution of time gaps δ between all interactions (Full data), and between target interaction and previous one for the First (left) and all others (right) target options for GTS on Zvuk dataset.

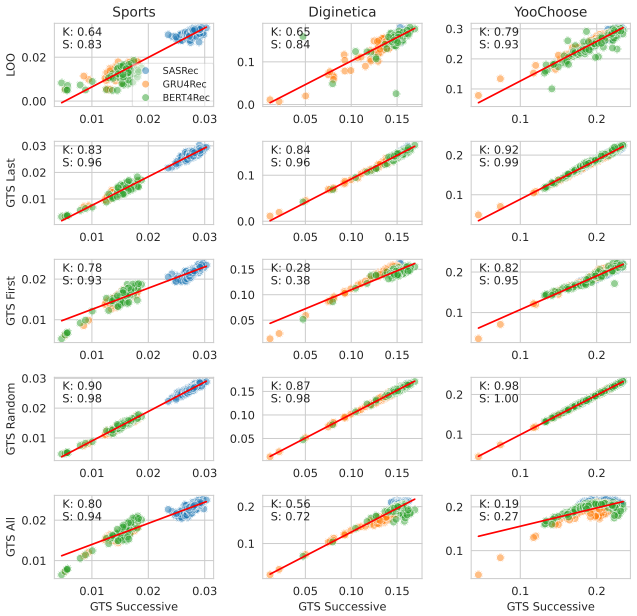


Figure 5: Scatterplots for NDCG@10 between GTS Sucv. target and other options. *K* and *S* denote Kendall and Spearman.

Table 6: Mean (across datasets) correlations between test GTS Successive target and other options for different metrics. Best values are in bold, second best are underlined.

Test Split	Kendall			Spearman		
	HR@10	MRR@10	NDCG@10	HR@10	MRR@10	NDCG@10
LOO	0.71	0.70	0.71	0.87	0.86	0.87
GTS Last	<u>0.83</u>	<u>0.82</u>	<u>0.83</u>	<u>0.93</u>	<u>0.94</u>	<u>0.94</u>
GTS First	0.70	0.60	0.62	0.82	0.70	0.72
GTS Random	**0.91**	**0.90**	**0.91**	**0.98**	**0.98**	**0.98**
GTS All	0.57	0.37	0.43	0.68	0.46	0.53

NDCG@10 on several datasets. The highest correlation is observed for GTS Random. GTS Last also shows a high, though slightly lower, correlation. In contrast, LOO, GTS First, and GTS All exhibit much greater dispersion across all datasets.

4.3.2 Correlation at different values of K. Figure 6 shows Kendall correlation across different values of K for NDCG on all datasets. To improve clarity, we omit GTS All from the plots due to its much lower correlation.

The relative order between splits remains fairly stable across different values of K. GTS Random shows the highest agreement with GTS Successive split, which is expected given their structural similarity. GTS Last typically follows with slightly lower, but still strong, correlation. In contrast, LOO and GTS First consistently show lower correlation, with significant drops on some datasets.

An exception is observed on MovieLens datasets: GTS Last is not better than LOO on ML-1M, and performs noticeably worse on ML-20M. Still, GTS Random outperforms LOO on both, suggesting that these datasets may have biased distributions for the last interaction in user histories.

4.3.3 Aggregated results. Table 6 presents Kendall and Spearman coefficients for HR@10, MRR@10, and NDCG@10 averaged across

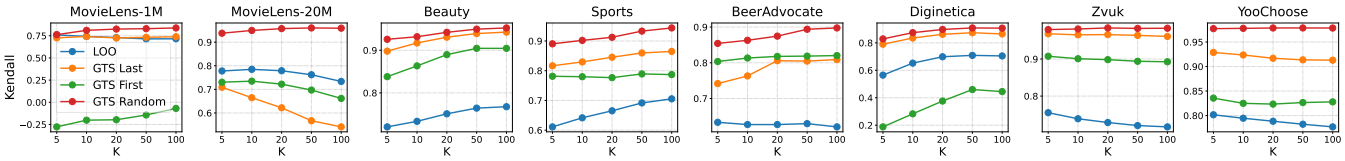


Figure 6: Kendall correlation between test NDCG@K for GTS with Successive target and other options.

all datasets. The different metrics and correlation types show consistent trends. GTS Random achieves the highest average correlation, with GTS Last slightly behind. LOO performs significantly worse, then goes GTS First, while GTS All shows the lowest correlation, highlighting the task mismatch for this target.

To summarize, we conclude that GTS Last and GTS Random are suitable options for the evaluation of sequential recommendation models. Both can serve as reasonable alternatives to the more computationally expensive GTS Successive strategy. GTS Random shows the highest correlation, but it is non-deterministic, which can lead to reproducibility issues unless the exact splits are stored. Alternatively, GTS Random can be run multiple times with different seeds to obtain more stable average results and an estimate of metrics variability. The very low correlation of GTS All confirms that it significantly deviates from the next-item prediction objective. GTS First also proves to be a less correlated target, consistent with the analysis presented in Section 4.2. Finally, the experimental results support the assumption of the limited alignment of the commonly used LOO split with a close-to-reality evaluation protocol.

4.4 Model rankings across different splits (RQ4)

Accurate final model ranking is crucial for both reliable research outcomes and practical deployment decisions. Although our correlation analysis (Section 4.2.1) shows that model rankings vary depending on the data split, those shifts may be primarily related to lower-ranked models rather than the best-performing configurations. In this section, we analyze the consistency of best model rankings across different splitting strategies [41].

For each split and target type, we sort models by their best test performance and then track how their positions change across splits and GTS targets. For this analysis, we also include *sequential item-based kNN (SeqKNN)* [33, 38] (a non-neural baseline) to better illustrate shifts in rankings. Figure 7 shows rankings by best test NDCG@10 under the LOO split, and the GTS split with GT validation and different test targets. We observe that model orderings *regularly shift* when splits and GTS targets change, revealing unstable rankings. For example, SASRec$^+$, which ranks first under the Successive target on ML-1M, falls to last place when evaluated under the All target option. Such inconsistency holds across most datasets, except for Amazon Beauty and Sports, where rankings remain stable across evaluation targets. Overall, SASRec$^+$ demonstrates the strongest performance on average, while GRU4Rec and SeqKNN frequently occupy the lowest positions. Among different evaluation targets, Last, Random, and Successive yield comparable rankings, closely matched by LOO. In contrast, First and All produce noticeably different model orders. Similar patterns hold across different test target options in alternative GTS validation setups (UB, LTI) and for different metrics. In summary, our findings confirm that *the choice of split and target may introduce significant ranking instability*. Careful selection of evaluation protocols is therefore essential for fair and reproducible comparisons.

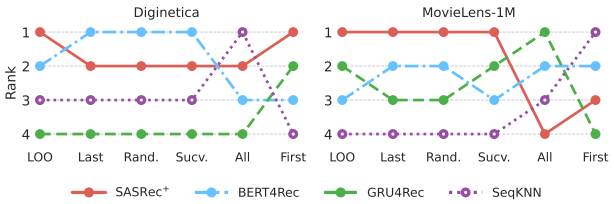


Figure 7: Model rankings based on test NDCG@10 for LOO split, and GTS split with global temporal validation.

Table 7: Mean (across datasets) correlations between test and validation metrics for GTS with (a) *Last* and (b) *Successive* test targets and different validation types. Best values are in bold, second best are underlined.

Correlation		Kendall			Spearman		
Target	Valid. Type↓	HR@10	MRR@10	NDCG@10	HR@10	MRR@10	NDCG@10
(a) Test Last	UB	0.72	0.72	0.74	0.87	0.88	0.89
	LTI	0.73	0.75	0.75	0.88	0.90	0.90
	GT Last	**0.78**	**0.79**	**0.79**	**0.93**	**0.93**	**0.93**
	GT First	0.61	0.54	0.57	0.77	0.69	0.73
	GT Rand.	0.75	0.75	0.76	0.90	0.91	0.92
	GT Sucv.	0.76	0.77	0.77	0.91	0.92	0.92
	GT All	0.46	0.37	0.43	0.59	0.50	0.56
(b) Test Sucv.	UB	0.78	0.78	0.80	0.93	0.92	0.94
	LTI	0.80	**0.83**	0.82	0.94	**0.95**	**0.95**
	GT Last	0.81	0.81	0.82	0.94	0.94	**0.95**
	GT First	0.64	0.56	0.59	0.80	0.72	0.75
	GT Rand.	0.80	0.81	0.81	0.94	0.94	0.94
	GT Sucv.	**0.83**	**0.83**	**0.83**	**0.95**	**0.95**	**0.95**
	GT All	0.48	0.37	0.44	0.60	0.49	0.56

4.5 Validation strategies for GTS (RQ5)

While the validation choice for the LOO split is straightforward, the GTS split allows for multiple validation strategies, as described in Section 3. To compare these strategies, we follow the same approach as in Section 4.2.1, but now we examine how well the validation metrics align with the corresponding test metrics under a given split. If the correlation between validation and test performance is low, the validation method is unreliable, as it may lead to selecting a suboptimal model.

4.5.1 Correlation at different values of K. Figure 8 shows the Kendall correlation between test and validation NDCG at various K across all datasets. The results are reported for GTS Last test split with three validation types: UB (user-based), LTI (last train item), and GT Last (global temporal with the last item as the target). For 5 out of 8 datasets, GT Last validation has consistently higher correlations with test metrics. On the Sports dataset, LTI validation performs on par with GT Last; on Youchoose, UB validation performs similarly

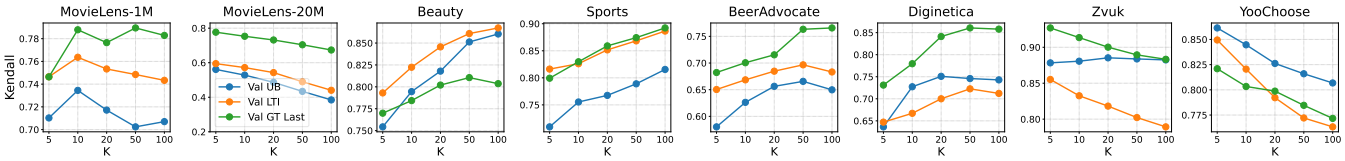


Figure 8: Kendall correlation between test and validation NDCG@K for GTS Last split with different validation strategies.

to GT Last. On the Beauty dataset, both LTI and UB options show higher correlations than GT Last.

4.5.2 Aggregated results. We also compute the average correlation across all datasets for HR@10, MRR@10, and NDCG@10 similarly to Section 4.3.3. Table 7 reports the average Kendall and Spearman correlations for two test split types (GTS Last and GTS Successive) and all considered validation strategies. As expected, GTS test splits align best with the appropriate global temporal validation types (GT Last and GT Successive). The All and First validation targets perform the worst, for the same reasons discussed in the analysis of test splits. It is worth noting that LTI and UB validation lag only slightly behind global temporal options. However, they share similar drawbacks with the LOO split, as described in Section 4.1.3, and should therefore be used with caution. Another observation is that for the GTS Successive test split, the GT Last validation performs nearly as well as GT Successive, suggesting that the simpler GT Last validation can be used without a significant loss in reliability.

In summary, the experimental results suggest that under global temporal splitting, the most reliable validation strategy is the corresponding global temporal validation. However, the GT validation comes with a limitation: the most recent data is not used for training, as it is reserved for validation. As a result, the test performance can become lower compared to other validation strategies. The following section analyses the impact of retraining.

4.6 Model retraining on combined data (RQ6)

Retraining the optimal model on combined training and validation data adds complexity to the pipeline, but is essential to deliver peak performance at industrial deployment. However, academic studies often omit retraining or leave it unreported, raising questions about the consistency of the results with real-world scenarios.

We select the best model on validation and compare the corresponding test metrics with and without retraining. Table 8 shows results for *Successive* and *Last* target options; other targets follow similar trends. Without retraining, UB consistently outperforms GT and LTI on the test set. We then examine the relative change in test metrics after retraining. The GT setup stably shows the largest improvement (e.g. 0.022 → 0.040: +81.8% on Beauty for *Successive*), as the retrained model captures shifts in user preferences over a long validation period, and benefits from additional training data. For UB, on average, retraining yields a modest improvement, while LTI and LOO experience more frequent performance drops. After retraining, *GT and UB achieve similar absolute test scores* (e.g. 0.160 vs. 0.158 on Diginetica for *Successive*), whereas LTI regularly remains lower. We also perform correlation analysis, which, for both GT and UB, shows high Kendall's τ (0.6–0.9) and Spearman (0.7–1.0) between *Test* and *Test R.* metrics, indicating *strong consistency for base and retrained setups* in selecting the same optimal models. In contrast, LTI generally shows lower values (0.4–0.6 and 0.5–0.7).

Table 8: Validation and test NDCG@10 of SASRec$^+$ at optimal validation configuration for different splits. *Test R.* denotes setup with retraining on combined training and validation data. *LTI* and *UB* in this study use only *Last* validation target.

Dataset		Diginetica				Amazon Beauty			
Split	**Target↓**	**Valid**	**Test**	**Test R.**	**Δ Test**	**Valid**	**Test**	**Test R.**	**Δ Test**
GT	Last	0.154	0.154	0.161	4.55%	0.046	0.024	0.037	54.2%
	Sucv.	0.154	0.149	0.160	7.38%	0.044	0.022	0.040	81.8%
UB	Last	0.180	0.152	0.155	1.97%	0.074	0.036	0.037	2.78%
	Sucv.	–	0.159	0.158	-0.63%	–	0.040	0.040	0.00%
LTI	Last	0.187	0.135	0.126	-6.67%	0.067	0.031	0.036	16.1%
	Sucv.	–	0.147	0.129	-12.2%	–	0.036	0.039	8.33%
LOO	Last	0.179	0.181	0.157	-13.3%	0.073	0.059	0.065	10.2%

Thus, when using GTS with GT or UB validation, retraining on combined training and validation data is important for achieving optimal deployment performance. For academic comparisons, retraining is still recommended, but it does not substantially alter the relative ranking of models.

5 Conclusion

We systematically compared leave-one-out and global temporal splitting strategies with various validation types and evaluation targets for sequential recommendations. Our experiments show that the common leave-one-out split, besides allowing for the emergence of temporal leakage and criticism raised in previous studies, demonstrates lower correlation with real-world evaluation scenarios and can distort model rankings. We also proved that the GTS All target option suffers from a task mismatch with standard next-item prediction, and that GTS First exhibits lower correlation with more realistic evaluation strategies due to significant shifts in time-gap distributions between interactions. In contrast, GTS with Last or Random target yields strong agreement with the more comprehensive but close-to-reality Successive evaluation scheme. To summarize, we conclude that global temporal split with Last, Random, and Successive targets are appropriate options for the evaluation of sequential recommendation models, with the Last and Random being reasonable alternatives to the more computationally expensive Successive strategy.

We further demonstrated that using a matching global temporal validation split produces reliable model selection, and that retraining on the combined training and validation data boosts final test performance for the reasonable validation options, compared to results for unretrained models.

Acknowledgments

We thank Fedor Dergachev for providing auxiliary code for the experiments and visualization.

References

[1] Vito Walter Anelli, Alejandro Bellogin, Antonio Ferrara, Daniele Malitesta, Felice Antonio Merra, Claudio Pomo, Francesco Maria Donini, and Tommaso Di Noia. 2021. Elliot: A Comprehensive and Rigorous Framework for Reproducible Recommender Systems Evaluation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Virtual Event, Canada) *(SIGIR '21)*. Association for Computing Machinery, New York, NY, USA, 2405–2414. doi:10.1145/3404835.3463245

[2] David Ben-Shimon, Alexander Tsikinovsky, Michael Friedmann, Bracha Shapira, Lior Rokach, and Johannes Hoerle. 2015. RecSys Challenge 2015 and the YOOCHOOSE Dataset. 357–358. doi:10.1145/2792838.2798723

[3] Rocío Cañamares and Pablo Castells. 2020. On Target Item Sampling in Offline Recommender System Evaluation. 259–268. doi:10.1145/3383313.3412259

[4] Huiyuan Chen, Yusan Lin, Menghai Pan, Lan Wang, Chin-Chia Michael Yeh, Xiaoting Li, Yan Zheng, Fei Wang, and Hao Yang. 2022. Denoising Self-attentive Sequential Recommendation.

[5] Junyi Chen, Lu Chi, Bingyue Peng, and Zehuan Yuan. 2024. Hllm: Enhancing sequential recommendations via hierarchical large language models for item and user modeling. *arXiv preprint arXiv:2409.12740* (2024).

[6] Ziqiang Cui, Haolun Wu, Bowei He, Ji Cheng, and Chen Ma. 2024. Diffusion-based Contrastive Learning for Sequential Recommendation. *arXiv preprint arXiv:2405.09369* (2024).

[7] Alexander Dallmann, Daniel Zoller, and Andreas Hotho. 2021. A Case Study on Sampling Strategies for Evaluating Neural Sequential Item Recommendation Models. In *Fifteenth ACM Conference on Recommender Systems (RecSys '21)*. ACM. doi:10.1145/3460231.3475943

[8] Janez Demšar. 2006. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research* 7, Jan (2006), 1–30.

[9] Hanwen Du, Hui Shi, Pengpeng Zhao, Deqing Wang, Victor S. Sheng, Yanchi Liu, Guanfeng Liu, and Lei Zhao. 2022. Contrastive Learning with Bidirectional Transformers for Sequential Recommendation. arXiv:2208.03895 [cs.IR]

[10] Xinyan Fan, Zheng Liu, Jianxun Lian, Wayne Zhao, Xing Xie, and Ji-Rong Wen. 2021. Lighter and Better: Low-Rank Decomposed Self-Attention Networks for Next-Item Recommendation. 1733–1737. doi:10.1145/3404835.3462978

[11] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems* (Copenhagen, Denmark) *(RecSys '19)*. Association for Computing Machinery, New York, NY, USA, 101–109. doi:10.1145/3298689.3347058

[12] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of the 13th ACM Conference on Recommender Systems* (Copenhagen, Denmark) *(RecSys '19)*. Association for Computing Machinery, New York, NY, USA, 101–109. doi:10.1145/3298689.3347058

[13] Hamed Firooz, Maziar Sanjabi, Adrian Englhardt, Aman Gupta, Ben Levine, Dre Olgiati, Gungor Polatkan, Iuliia Melnychuk, Karthik Ramgopal, Kirill Talanine, et al. 2025. 360brew: A decoder-only foundation model for personalized ranking and recommendation. *arXiv preprint arXiv:2501.16450* (2025).

[14] Evgeny Frolov, Tatyana Matveeva, Leyla Mirvakhabova, and Ivan Oseledets. 2024. Self-Attentive Sequential Recommendations with Hyperbolic Representations. (2024).

[15] Scott Graham, Jun-Ki Min, and Tao Wu. 2019. Microsoft recommenders: tools to accelerate developing recommender systems. In *Proceedings of the 13th ACM Conference on Recommender Systems* (Copenhagen, Denmark) *(RecSys '19)*. Association for Computing Machinery, New York, NY, USA, 542–543. doi:10.1145/3298689.3346967

[16] Asela Gunawardana, Guy Shani, and Sivan Yogev. 2012. Evaluating recommender systems. In *Recommender systems handbook*. Springer, 547–601.

[17] Danil Gusak, Gleb Mezentsev, Ivan Oseledets, and Evgeny Frolov. 2024. RECE: Reduced Cross-Entropy Loss for Large-Catalogue Sequential Recommenders. *Proceedings of 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*. doi:10.1145/3627673.3679986

[18] F Maxwell Harper and Joseph A Konstan. 2015. The movielens datasets: History and context. *Acm transactions on interactive intelligent systems (tiis)* 5, 4 (2015), 1–19.

[19] Jesse Harte, Wouter Zorgdrager, Panos Louridas, Asterios Katsifodimos, Dietmar Jannach, and Marios Fragkoulis. 2023. Leveraging large language models for sequential recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1096–1102.

[20] Ruining He, Wang-Cheng Kang, and Julian McAuley. 2017. Translation-based recommendation. In *Proceedings of the eleventh ACM conference on recommender systems*. 161–169.

[21] Ruining He and Julian McAuley. 2016. Fusing similarity models with markov chains for sparse sequential recommendation. In *2016 IEEE 16th international conference on data mining (ICDM)*. IEEE, 191–200.

[22] Balázs Hidasi and Ádám Tibor Czapp. 2023. The effect of third party implementations on reproducibility. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 272–282.

[23] Balázs Hidasi and Ádám Tibor Czapp. 2023. Widespread Flaws in Offline Evaluation of Recommender Systems. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 848–855.

[24] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).

[25] Karl Higley, Even Oldridge, Ronay Ak, Sara Rabhi, and Gabriel de Souza Pereira Moreira. 2022. Building and Deploying a Multi-Stage Recommender System with Merlin. In *Proceedings of the 16th ACM Conference on Recommender Systems* (Seattle, WA, USA) *(RecSys '22)*. Association for Computing Machinery, New York, NY, USA, 632–635. doi:10.1145/3523227.3551468

[26] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (Nov. 1997), 1735–1780. doi:10.1162/neco.1997.9.8.1735

[27] Dietmar Jannach, Massimo Quadrana, and Paolo Cremonesi. 2022. *Session-Based Recommender Systems*. Springer US, New York, NY, 301–334. doi:10.1007/978-1-0716-2197-4_8

[28] Yitong Ji, Aixin Sun, Jie Zhang, and Chenliang Li. 2023. A critical study on data leakage in recommender system offline evaluation. *ACM Transactions on Information Systems* 41, 3 (2023), 1–27.

[29] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[30] Anton Klenitskiy and Alexey Vasilev. 2023. Turning dross into gold loss: is bert4rec really better than sasrec?. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1120–1125.

[31] Anton Klenitskiy, Anna Volodkevich, Anton Pembek, and Alexey Vasilev. 2024. Does It Look Sequential? An Analysis of Datasets for Evaluation of Sequential Recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 1067–1072.

[32] Walid Krichene and Steffen Rendle. 2020. On Sampled Metrics for Item Recommendation. In *KDD 2020*. https://dl.acm.org/doi/10.1145/3394486.3403226

[33] Sara Latifi, Dietmar Jannach, and Andres Ferraro. 2022. Sequential Recommendation: A Study on Transformers, Nearest Neighbors and Sampled Metrics. *Information Sciences* 609 (07 2022). doi:10.1016/j.ins.2022.07.079

[34] Jiayu Li, Hanyu Li, Zhiyu He, Weizhi Ma, Peijie Sun, Min Zhang, and Shaoping Ma. 2024. Rechorus2. 0: A modular and task-flexible recommendation library. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 454–464.

[35] Jiacheng Li, Yujie Wang, and Julian McAuley. 2020. Time Interval Aware Self-Attention for Sequential Recommendation. 322–330. doi:10.1145/3336191.3371786

[36] Jinming Li, Wentao Zhang, Tian Wang, Guanglei Xiong, Alan Lu, and Gerard Medioni. 2023. GPT4Rec: A generative framework for personalized recommendation and user interests interpretation. *arXiv preprint arXiv:2304.03879* (2023).

[37] Chang Liu, Xiaoguang Li, Guohao Cai, Zhenhua Dong, Hong Zhu, and Lifeng Shang. 2021. Non-invasive Self-attention for Side Information Fusion in Sequential Recommendation. arXiv:2103.03578 [cs.IR]

[38] Malte Ludewig and Dietmar Jannach. 2018. Evaluation of session-based recommendation algorithms. *User Modeling and User-Adapted Interaction* 28 (2018), 331–390.

[39] Julian McAuley, Jure Leskovec, and Dan Jurafsky. 2012. Learning attitudes and attributes from multi-aspect reviews. In *2012 IEEE 12th International Conference on Data Mining*. IEEE, 1020–1025.

[40] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. 2015. Image-based Recommendations on Styles and Substitutes. arXiv:1506.04757 [cs.CV]

[41] Zaiqiao Meng, Richard McCreadie, Craig Macdonald, and Iadh Ounis. 2020. Exploring data splitting strategies for the evaluation of recommendation models. In *Proceedings of the 14th acm conference on recommender systems*. 681–686.

[42] Gleb Mezentsev, Danil Gusak, Ivan Oseledets, and Evgeny Frolov. 2024. Scalable cross-entropy loss for sequential recommendations with large item catalogs. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 475–485.

[43] Lien Michiels, Robin Verachtert, and Bart Goethals. 2022. RecPack: An(other) Experimentation Toolkit for Top-N Recommendation using Implicit Feedback Data. In *Proceedings of the 16th ACM Conference on Recommender Systems* (Seattle, WA, USA) *(RecSys '22)*. Association for Computing Machinery, New York, NY, USA, 648–651. doi:10.1145/3523227.3551472

[44] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.

[45] Aleksandr Petrov and Craig Macdonald. 2022. A systematic review and replicability study of bert4rec for sequential recommendation. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 436–447.

[46] Aleksandr Vladimirovich Petrov and Craig Macdonald. 2023. gsasrec: Reducing overconfidence in sequential recommendation trained with negative sampling. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 116–128.

[47] Ruihong Qiu, Zi Huang, Hongzhi Yin, and Zijian Wang. 2022. Contrastive Learning for Representation Degeneration Problem in Sequential Recommendation. 813–823. doi:10.1145/3488560.3498433

[48] Mostafa Rahmani, James Caverlee, and Fei Wang. 2023. Incorporating time in sequential recommendation models. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 784–790.

[49] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.

[50] Noveen Sachdeva and Julian McAuley. 2020. How Useful are Reviews for Recommendation? A Critical Review and Potential Improvements. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval* (Virtual Event, China) *(SIGIR '20)*. Association for Computing Machinery, New York, NY, USA, 1845–1848. doi:10.1145/3397271.3401281

[51] Aghiles Salah, Quoc-Tuan Truong, and Hady W. Lauw. 2020. Cornac: a comparative framework for multimodal recommender systems. *J. Mach. Learn. Res.* 21, 1, Article 95 (Jan. 2020), 5 pages.

[52] Teresa Scheidt and Joeran Beel. 2021. Time-dependent Evaluation of Recommender Systems.. In *Perspectives@ RecSys*.

[53] Valeriy Shevchenko, Nikita Belousov, Alexey Vasilev, Vladimir Zholobov, Artyom Sosedka, Natalia Semenova, Anna Volodkevich, Andrey Savchenko, and Alexey Zaytsev. 2024. From variability to stability: Advancing RecSys benchmarking practices. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5701–5712.

[54] Anima Singh, Trung Vu, Nikhil Mehta, Raghunandan Keshavan, Maheswaran Sathiamoorthy, Yilin Zheng, Lichan Hong, Lukasz Heldt, Li Wei, Devansh Tandon, et al. 2024. Better generalization with semantic ids: A case study in ranking for recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 1039–1044.

[55] Aixin Sun. 2023. Take a fresh look at recommender systems from an evaluation standpoint. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2629–2638.

[56] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 1441–1450.

[57] Zhu Sun, Hui Fang, Jie Yang, Xinghua Qu, Hongyang Liu, Di Yu, Yew-Soon Ong, and Jie Zhang. 2023. DaisyRec 2.0: Benchmarking Recommendation for Rigorous Evaluation. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 7 (July 2023), 8206–8226. doi:10.1109/TPAMI.2022.3231891

[58] Zhu Sun, Di Yu, Hui Fang, Jie Yang, Xinghua Qu, Jie Zhang, and Cong Geng. 2020. Are We Evaluating Rigorously? Benchmarking Recommendation for Reproducible Evaluation and Fair Comparison. In *Proceedings of the 14th ACM Conference on Recommender Systems* (Virtual Event, Brazil) *(RecSys '20)*. Association for Computing Machinery, New York, NY, USA, 23–32. doi:10.1145/3383313.3412489

[59] Jiaxi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.

[60] Xuewen Tao, Mingming Ha, Qiongxu Ma, Hongwei Cheng, Wenfang Lin, Xiaobo Guo, Linxun Cheng, and Bing Han. 2023. Task aware feature extraction framework for sequential dependence multi-task learning. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 151–160.

[61] Alexey Vasilev, Anna Volodkevich, Denis Kulandin, Tatiana Bysheva, and Anton Klenitskiy. 2024. RePlay: a Recommendation Framework for Experimentation and Production Use. In *Proceedings of the 18th ACM Conference on Recommender Systems* (Bari, Italy) *(RecSys '24)*. Association for Computing Machinery, New York, NY, USA, 1191–1194. doi:10.1145/3640457.3691701

[62] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[63] Robin Verachtert, Lien Michiels, and Bart Goethals. 2022. Are We Forgetting Something? Correctly Evaluate a Recommender System With an Optimal Training Window.. In *Perspectives@ RecSys*.

[64] Anna Volodkevich, Danil Gusak, Anton Klenitskiy, and Alexey Vasilev. 2024. Autoregressive Generation Strategies for Top-K Sequential Recommendations. *arXiv preprint arXiv:2409.17730* (2024).

[65] Lukas Wegmeth, Tobias Vente, Lennart Purucker, and Joeran Beel. 2023. The Effect of Random Seeds for Data Splitting on Recommendation Accuracy.. In *Perspectives@ RecSys*.

[66] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2019. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771* (2019).

[67] Wujiang Xu, Zujie Liang, Jiaojiao Han, Xuying Ning, Wenfang Lin, Linxun Chen, Feng Wei, and Yongfeng Zhang. 2024. Slmrec: empowering small language models for sequential recommendation. *arXiv e-prints* (2024), arXiv–2405.

[68] Yufei Ye, Wei Guo, Jin Yao Chin, Hao Wang, Hong Zhu, Xi Lin, Yuyang Ye, Yong Liu, Ruiming Tang, Defu Lian, et al. 2025. FuXi-alpha: Scaling Recommendation Model with Feature Interaction Enhanced Transformer. *arXiv preprint arXiv:2502.03036* (2025).

[69] Xiaohan Yu, Li Zhang, Xin Zhao, and Yue Wang. 2024. Break the ID-Language Barrier: An Adaption Framework for Sequential Recommendation. *arXiv preprint arXiv:2411.18262* (2024).

[70] Huimin Zeng, Xiaojie Wang, Anoop Jain, Zhicheng Dou, and Dong Wang. 2025. A non-contrastive learning framework for sequential recommendation with preference preserving profile generation. (2025).

[71] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).

[72] Wayne Xin Zhao, Zihan Lin, Zhichao Feng, Pengfei Wang, and Ji-Rong Wen. 2022. A revisiting study of appropriate offline evaluation for top-N recommendation algorithms. *ACM Transactions on Information Systems* 41, 2 (2022), 1–41.

[73] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. 2021. RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management* (Virtual Event, Queensland, Australia) *(CIKM '21)*. Association for Computing Machinery, New York, NY, USA, 4653–4664. doi:10.1145/3459637.3482016

[74] Jieming Zhu, Jinyang Liu, Shuai Yang, Qi Zhang, and Xiuqiang He. 2021. Open Benchmarking for Click-Through Rate Prediction. In *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*, Gianluca Demartini, Guido Zuccon, J. Shane Culpepper, Zi Huang, and Hanghang Tong (Eds.). ACM, 2759–2769. doi:10.1145/3459637.3482486